Federated Learning Optimization V2

Early-Stopped, Fairness-Aware, Communication-Aware FL on UCI HAR

Author Name Placeholder

April 26, 2026

Abstract

This report presents the upgraded federated learning optimization project. The system trains human-activity recognition models on UCI HAR using each subject as a federated client. The upgraded study includes early stopping with a high maximum round cap, 10 random seeds, FedAvg, CVaR-style fairness, centralized and local baselines, grid and differential-evolution hyperparameter search, communication-budget LP duality, KKT diagnostics, classification metrics, calibration, non-IID analysis, client-level analysis, extensive visualizations, checkpoints, and reproducibility artifacts.

Contents

1 Executive Summary

The final v2 run used max rounds 1000, patience 30, min-delta 0.001, and 10 seeds. Early stopping means each run could stop before the maximum if loss stopped improving.

Method	Accuracy	Worst Client	Loss	Stopped Round
fedavg_default	0.9697	0.8912	0.0841	226.4
cvar_0	0.9695	0.8839	0.0845	223.4
cvar_0.5	0.9701	0.8740	0.0860	226.3
cvar_0.75	0.9675	0.8739	0.0943	231.1
cvar_0.9	0.9707	0.8764	0.0822	241.6
cvar_0.95	0.9708	0.8789	0.0818	241.6
centralized	0.9847	0.9407	0.0621	56.4
local_only	0.9878	0.9444	0.0410	50.0
grid_best_validated	0.9726	0.8827	0.0896	242.3
ga_best_validated	0.9654	0.8373	0.1178	134.7

Table 1: Method comparison with early stopping.

Best mean accuracy method: `local_only`. Default FedAvg reached 0.9697 mean accuracy and 0.8912 worst-client accuracy. Centralized training reached 0.9847. Grid-validated FedAvg reached 0.9726. GA-validated reached 0.9654 with lower communication than most full FedAvg variants.

2 Dataset and EDA

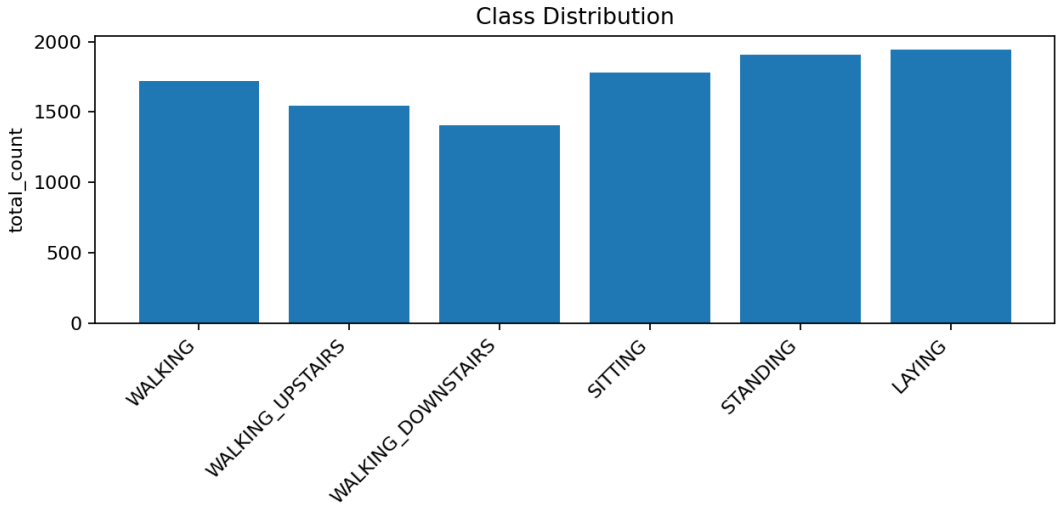

Figure 1: Activity class distribution.

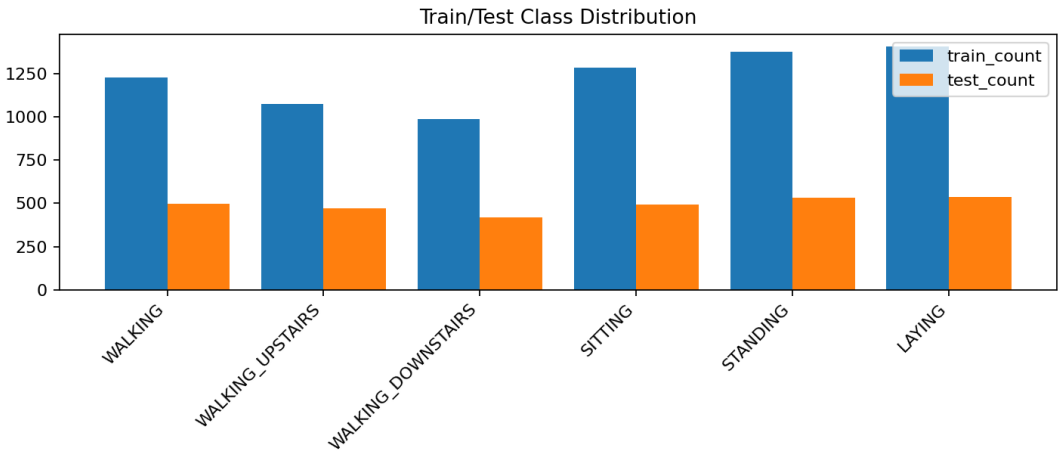

Figure 2: Train/test class distribution.

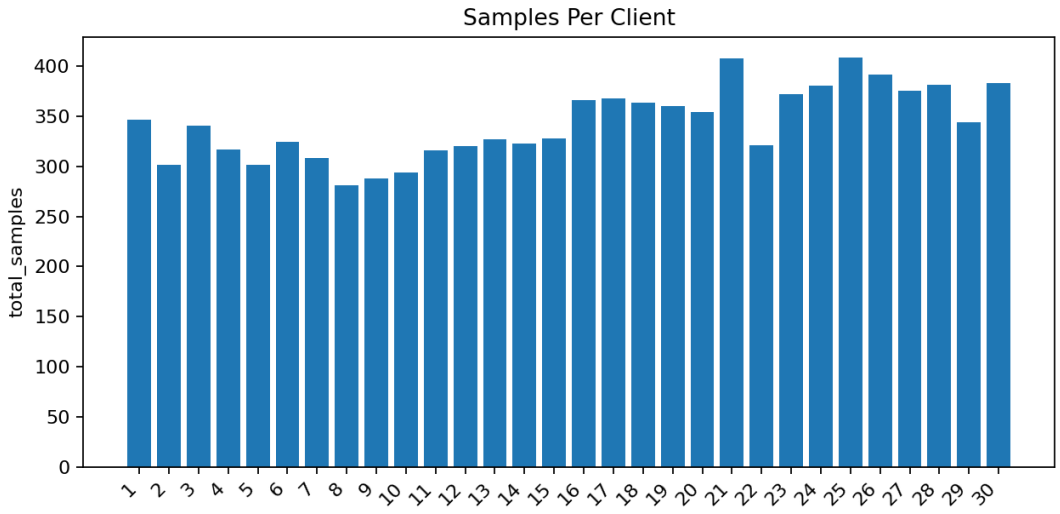

Figure 3: Samples per federated client.

Figure 4: Client-by-activity heatmap showing non-IID structure.

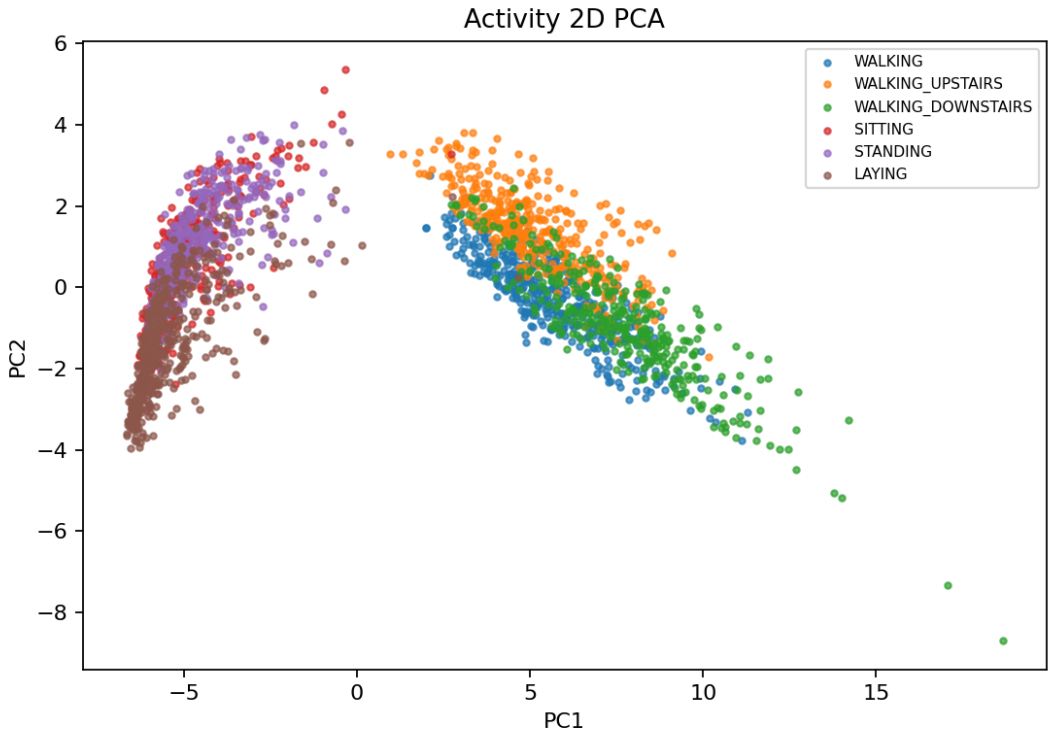

Figure 5: 2D PCA colored by activity.

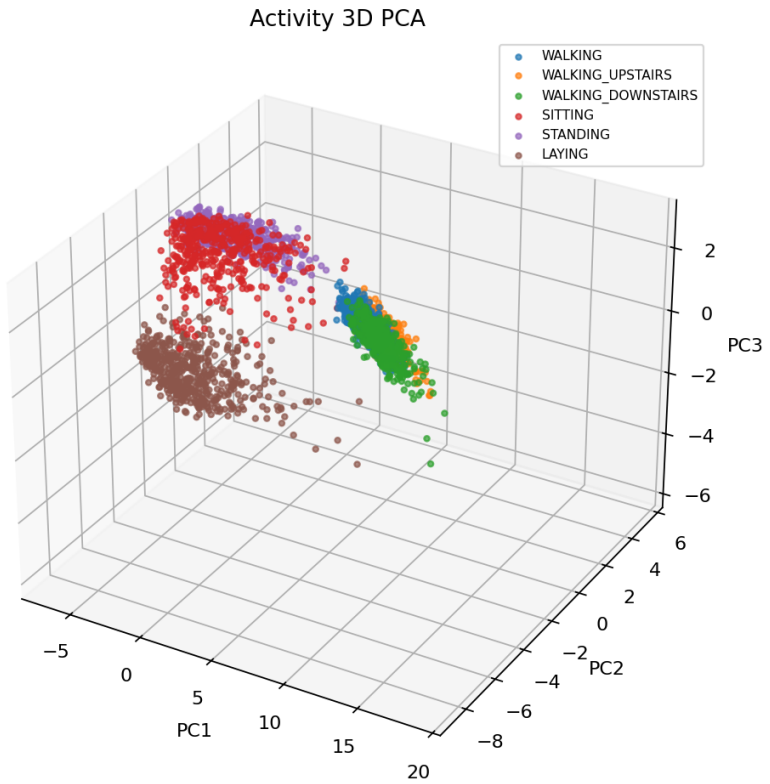

Figure 6: 3D PCA colored by activity.

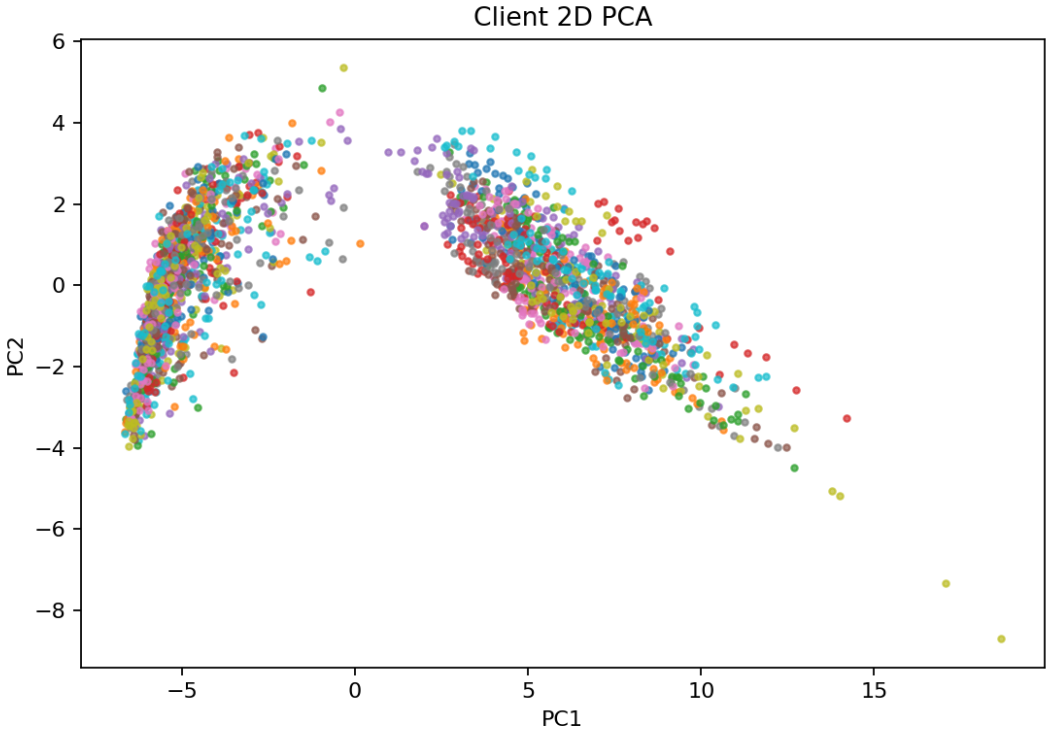

Figure 7: 2D PCA colored by client.

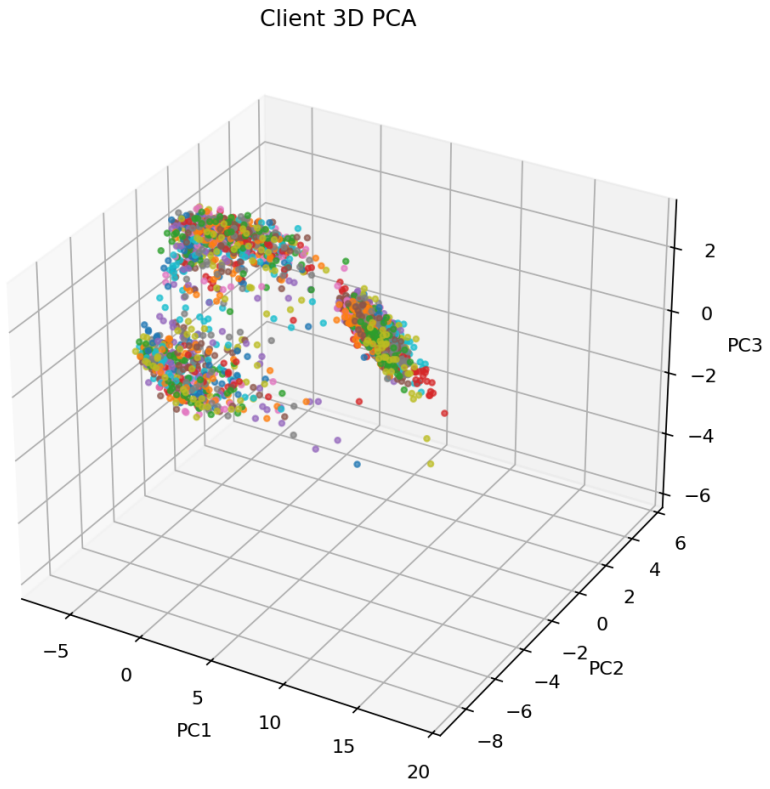

Figure 8: 3D PCA colored by client.

3 Training and Early Stopping

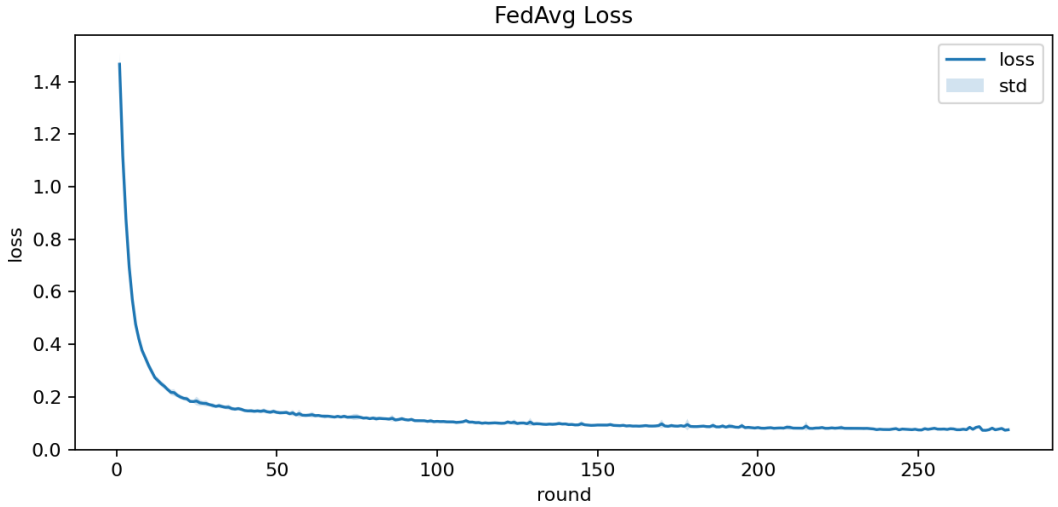

Figure 9: FedAvg loss across rounds with seed variation.

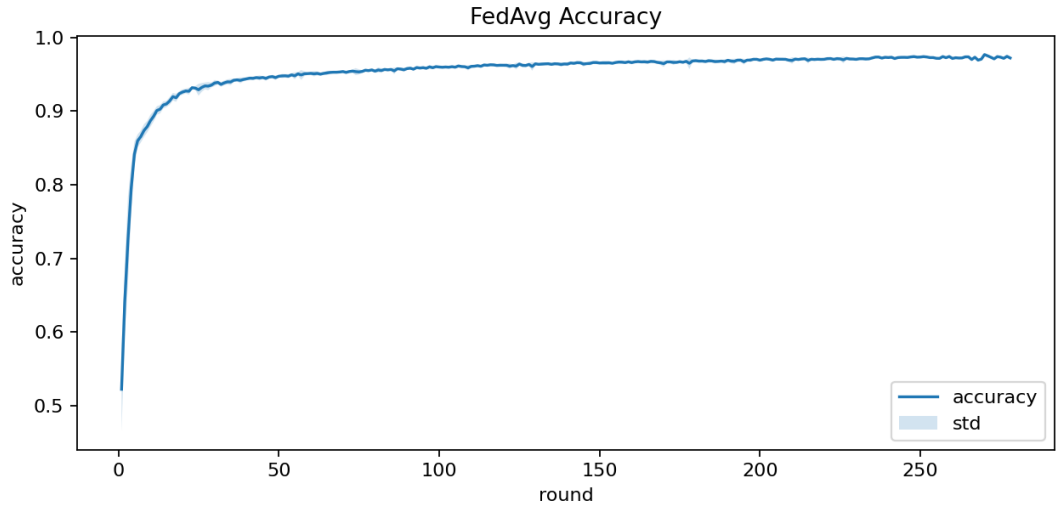

Figure 10: FedAvg accuracy across rounds.

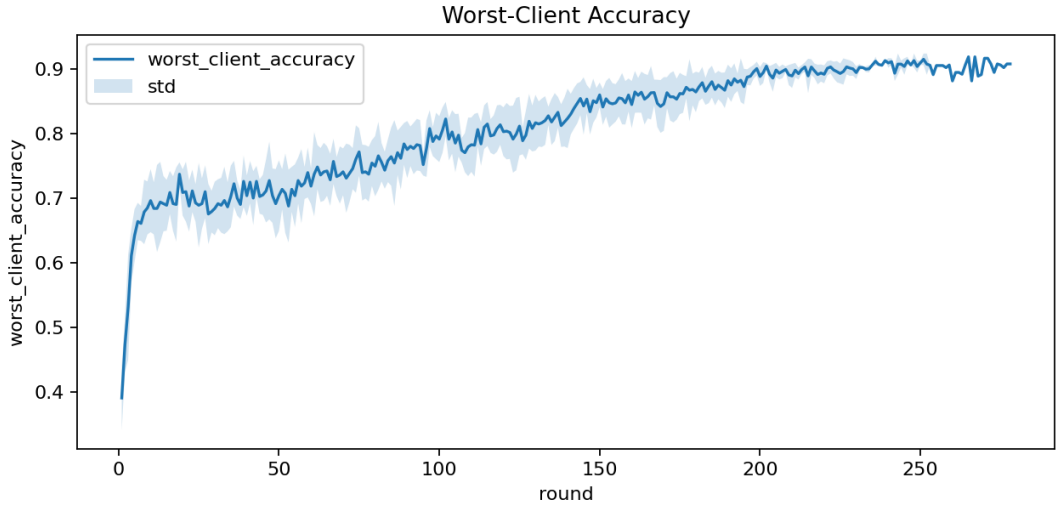

Figure 11: Worst-client accuracy across rounds.

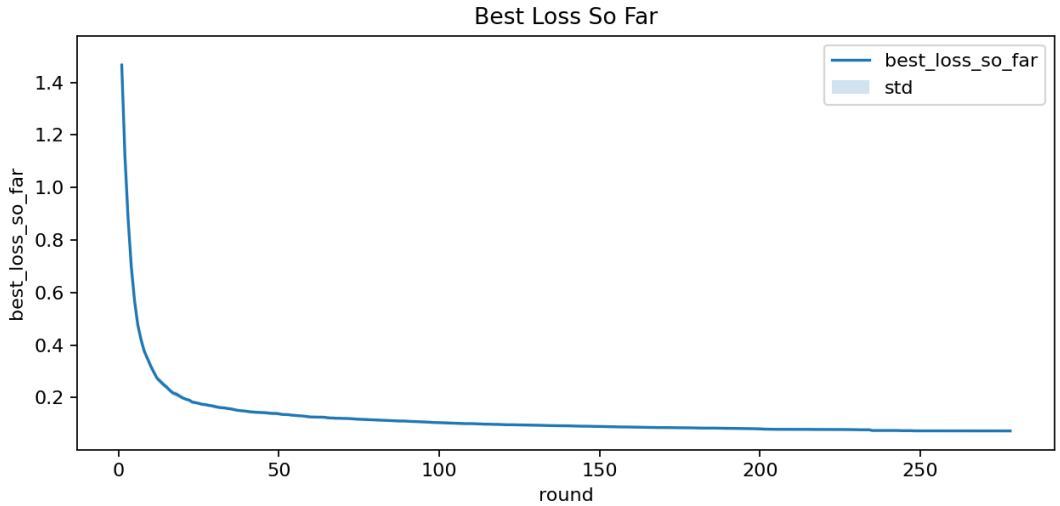

Figure 12: Best loss so far across rounds.

4 Classification Metrics

The final selected prediction artifact reports aggregate accuracy 0.9725, macro F1 0.9738, and weighted F1 0.9725.

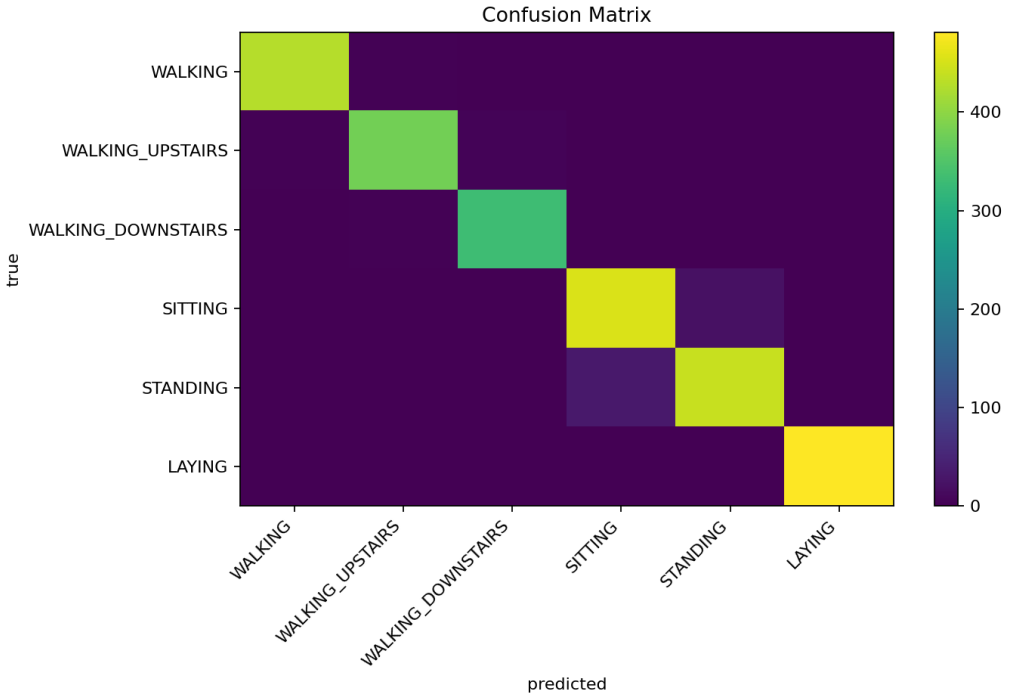

Figure 13: Raw confusion matrix.

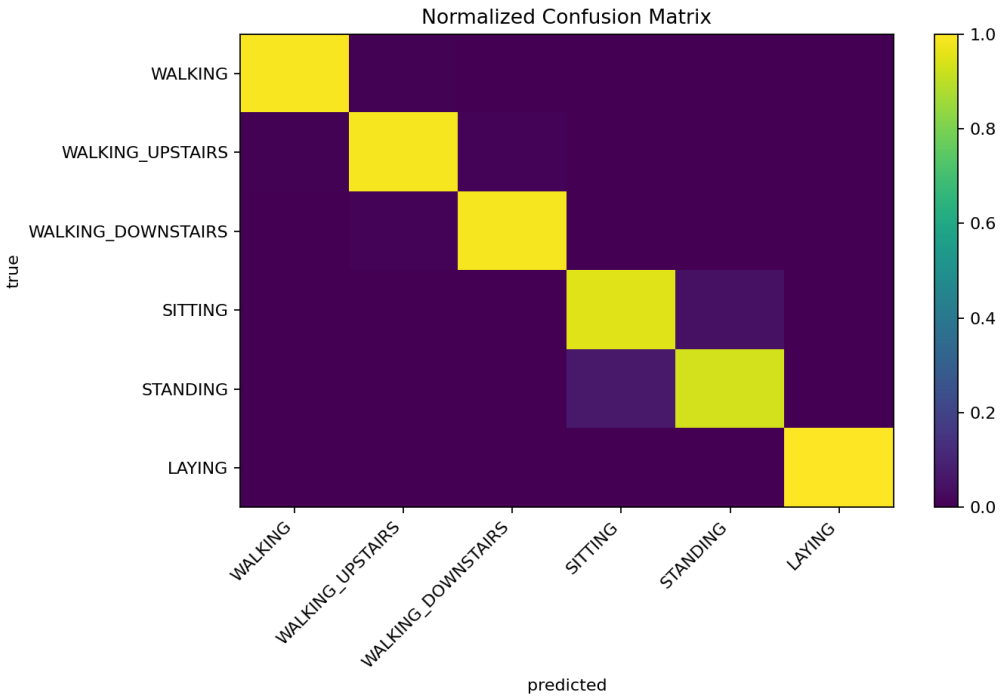

Figure 14: Normalized confusion matrix.

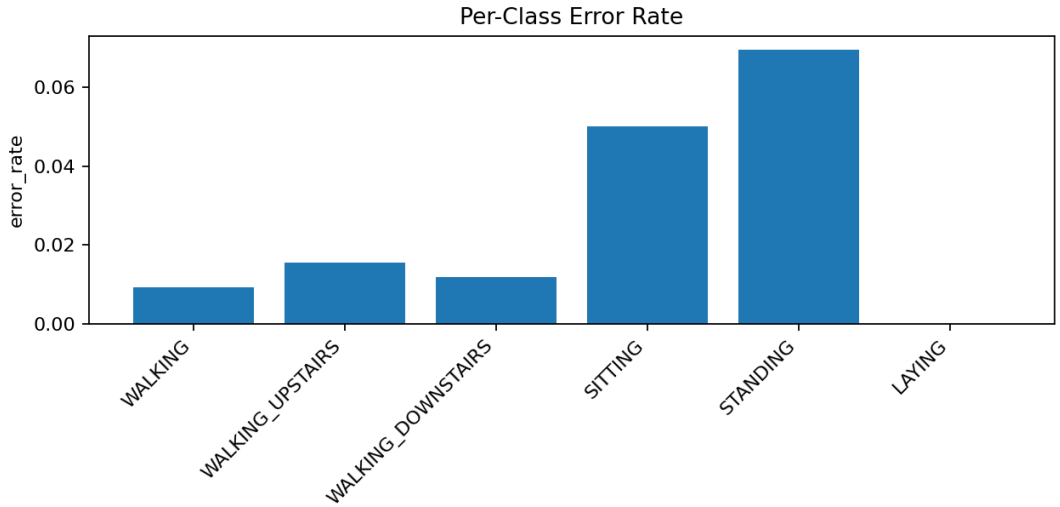

Figure 15: Per-class error rate.

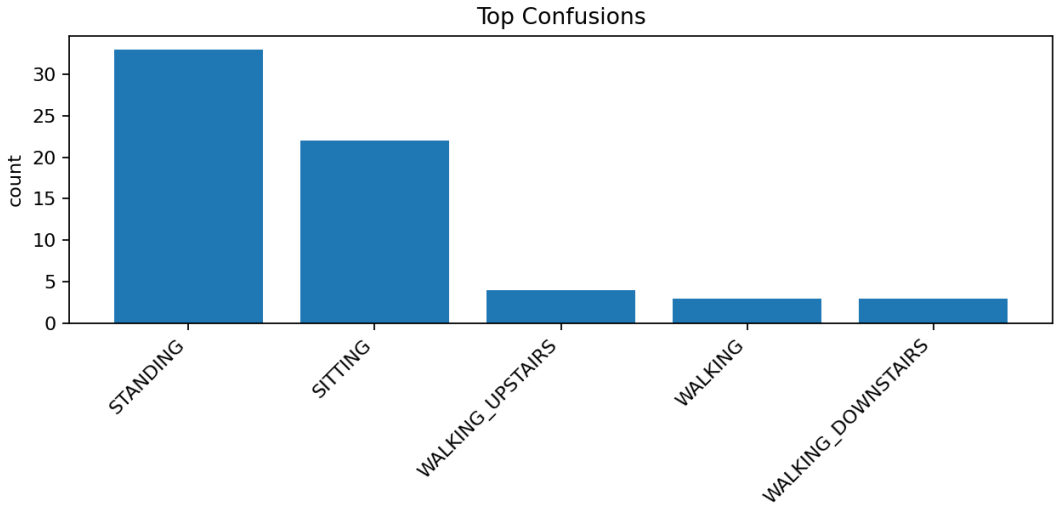

Figure 16: Top confused activity pairs.

5 Optimization and Search

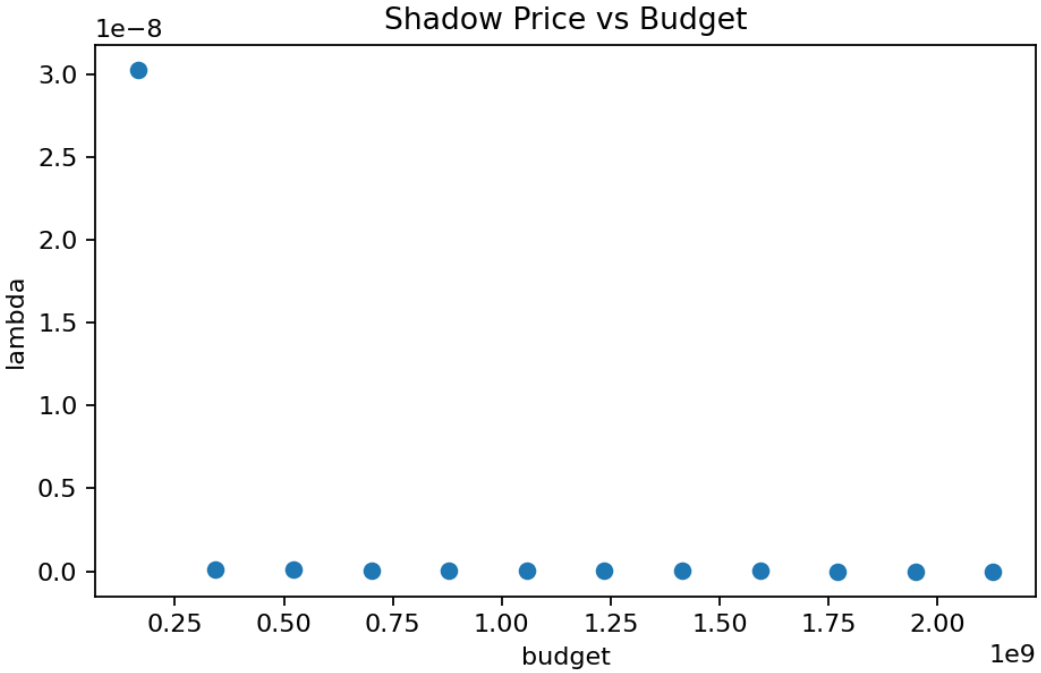

Figure 17: Communication budget versus LP shadow price.

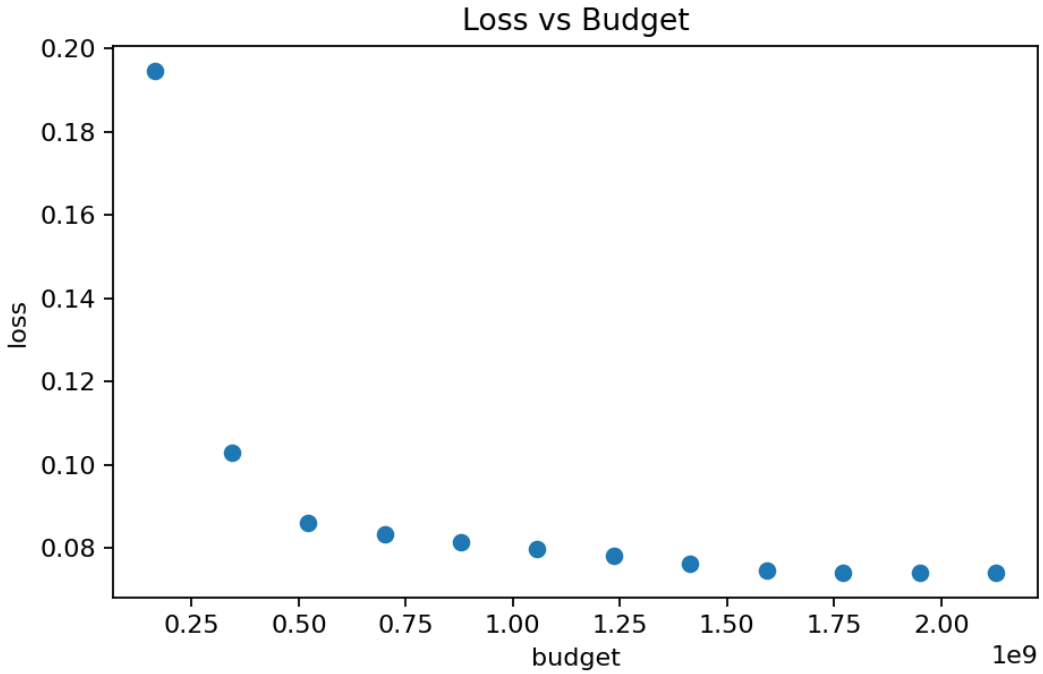

Figure 18: LP loss versus communication budget.

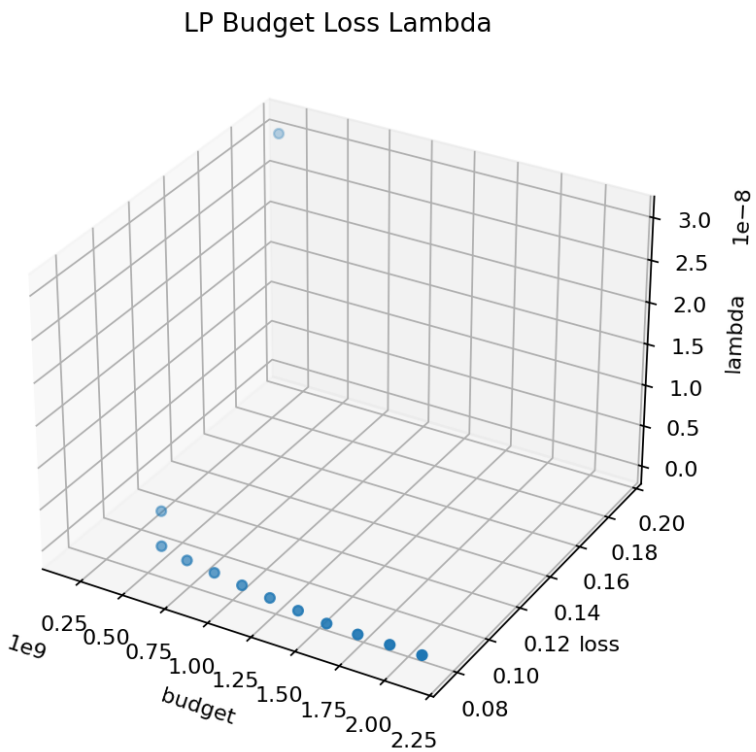

Figure 19: 3D LP budget, loss, and lambda plot.

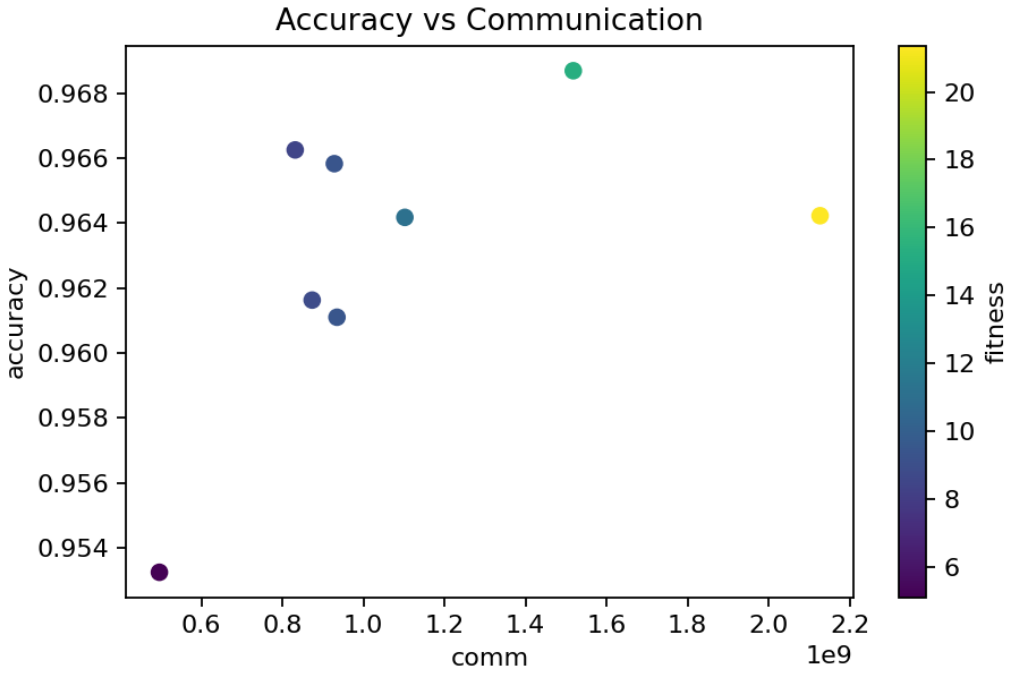

Figure 20: Search configurations: accuracy versus communication.

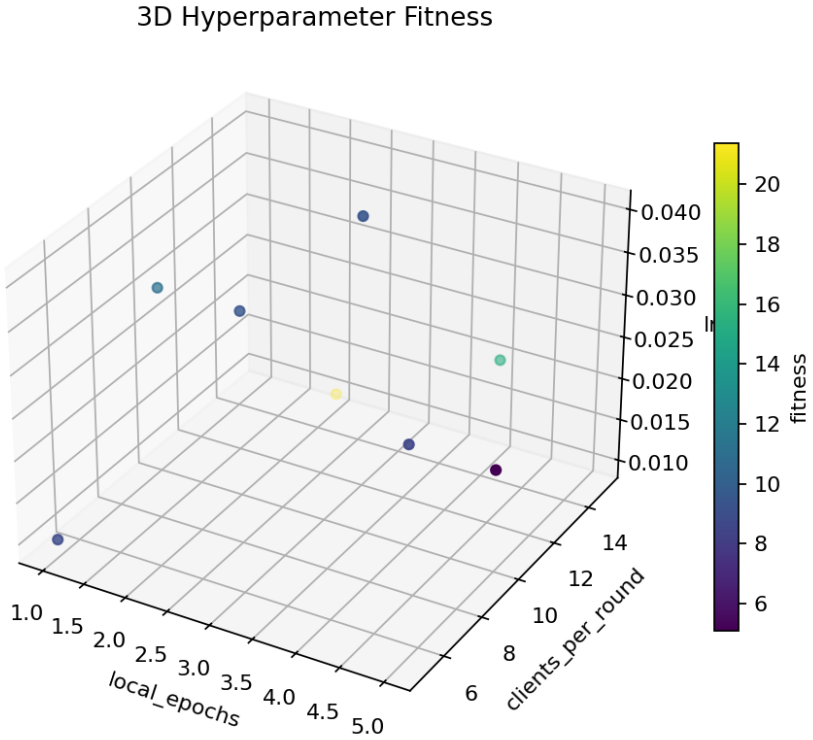

Figure 21: 3D hyperparameter search fitness.

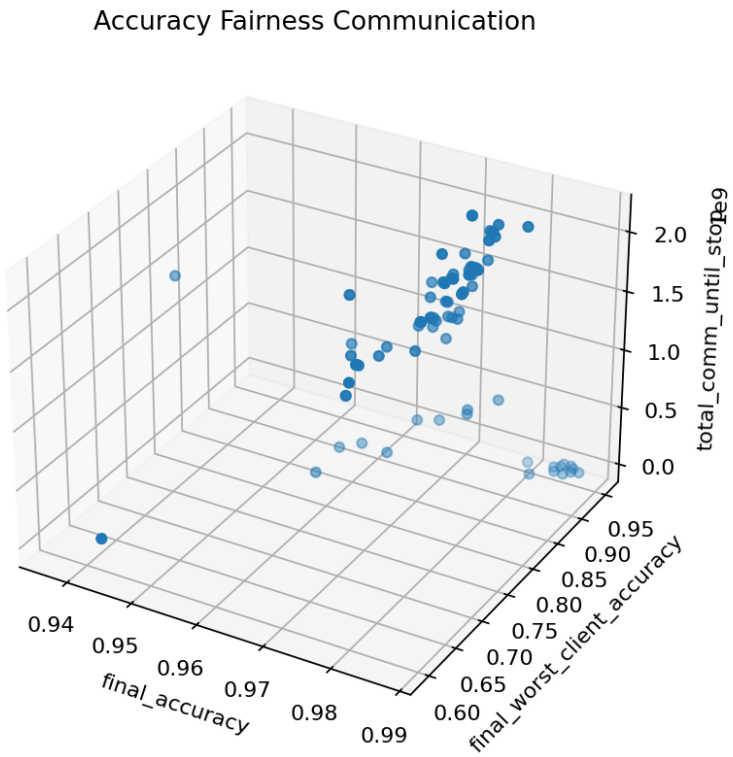

Figure 22: 3D accuracy, fairness, and communication tradeoff.

6 Baselines

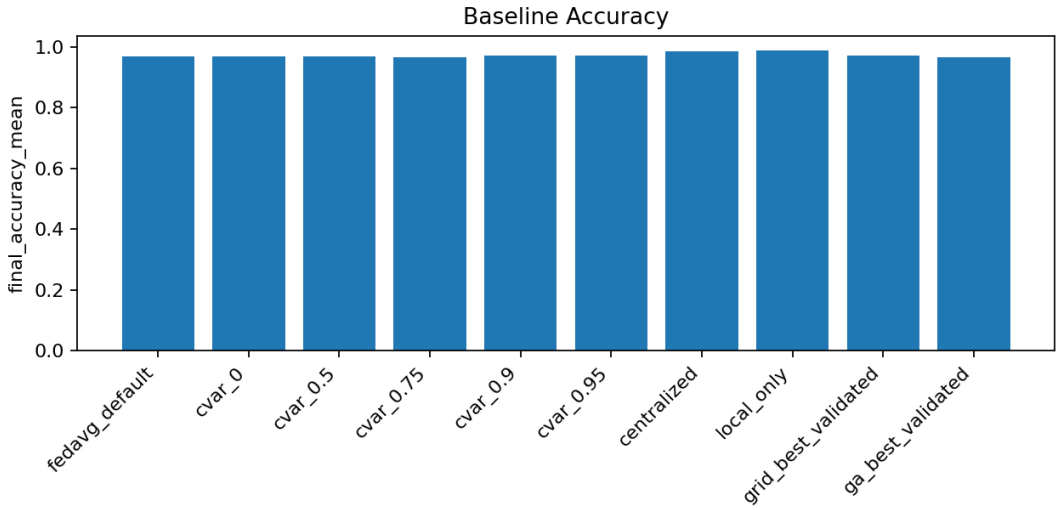

Figure 23: Baseline mean accuracy comparison.

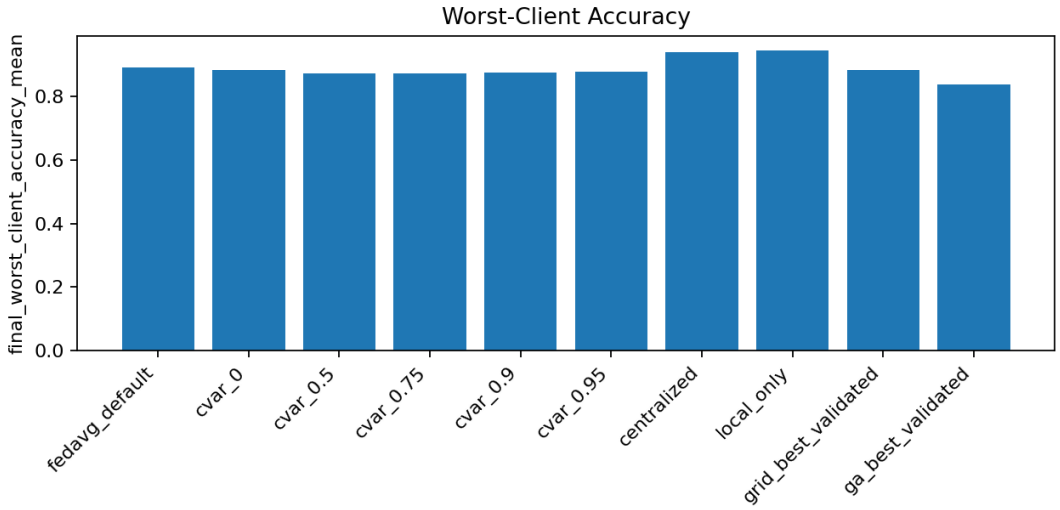

Figure 24: Worst-client accuracy by method.

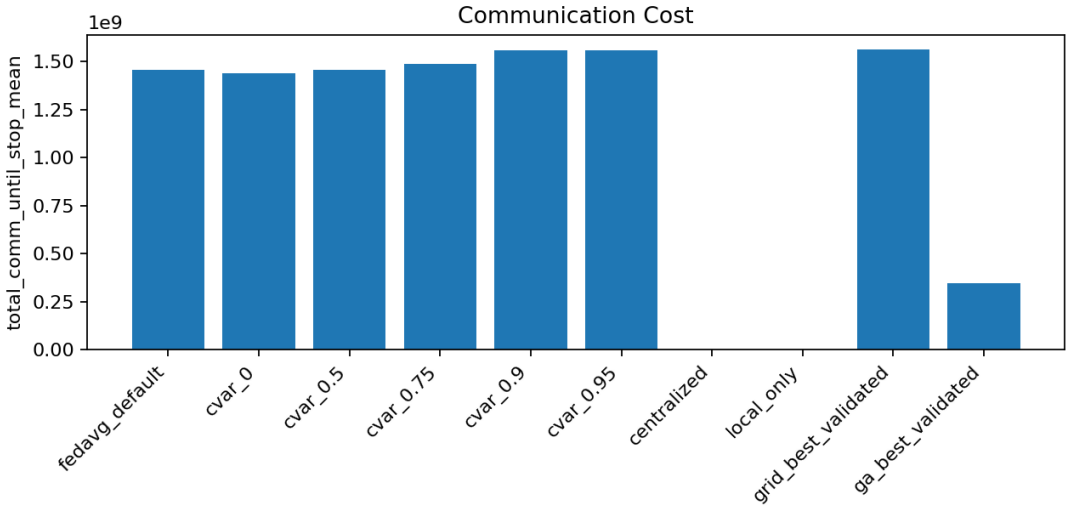

Figure 25: Communication cost by method.

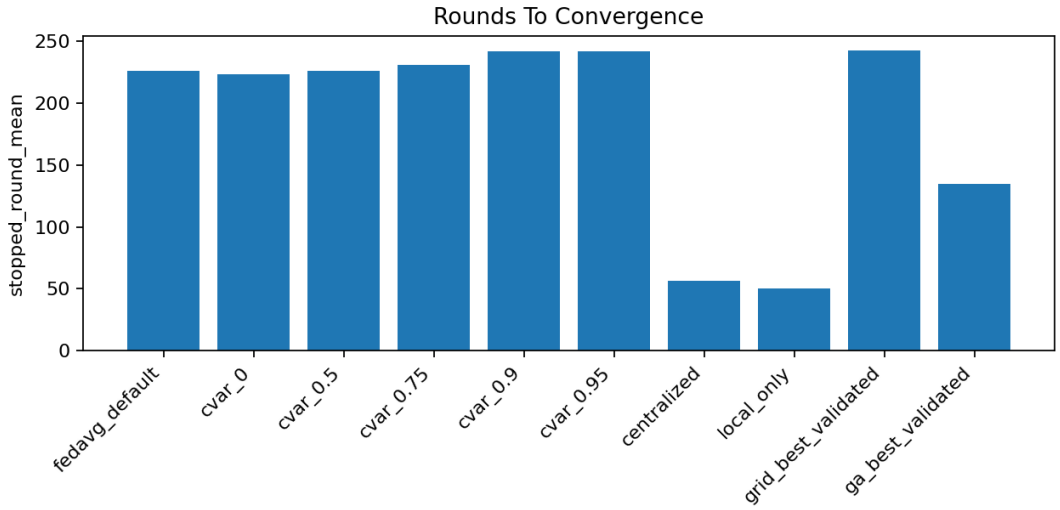

Figure 26: Average stopped round by method.

7 Statistical and Diagnostic Analysis

The v2 run saves confidence intervals, paired tests, effect sizes, non-IID metrics, calibration bins, communication efficiency metrics, failure-mode summaries, and theoretical alignment tables under `outputs/full_uci_v2/`.

8 Runtime

Stage	Seconds
load_data	0.30
eda	1.00
fedavg_default	190.35
cvar_sweep	978.71
baselines	68.55
search	2275.93
predictions_metrics	0.05
lp_diagnostics	0.19
diagnostics	0.62
plots	1.29
artifacts	0.01

Table 2: Runtime by stage.

9 Artifacts

The run saves model checkpoints for centralized, FedAvg, best CVaR, best grid, and best GA models. It also saves the scaler, predictions, classification reports, confusion matrices, LP outputs, search history, raw round metrics, EDA tables, and diagnostic analysis outputs.

10 Professor Feedback and Extensions

10.1 Improving the Communication Model

The current communication model is intentionally simple. It assumes that every selected client downloads the full global model and uploads the full locally trained model once per round. If the model has d parameters and each

parameter is sent as a 32-bit float, then one model transfer costs $4d$ bytes. For K_t selected clients in round t, the current round communication estimate is

$$C_t = 2K_t(4d),$$

where the factor of 2 accounts for one server-to-client download and one client-to-server upload. For the HARMLP model used here, $d = 80{,}582$, so one model copy is roughly 322,328 bytes.

This model makes several assumptions:

- All parameters are transmitted densely every round.
- Upload and download costs are symmetric.
- Network latency, protocol headers, serialization overhead, retransmission, and client availability are ignored.
- Secure aggregation, encryption, authentication, and privacy overhead are ignored.
- Client energy cost and wall-clock time are not modeled separately from byte count.
- Quantization, sparsification, delta updates, and compression are not used.

A more realistic communication model can be written as

$$C_t = \sum_{i \in S_t} \left(B_{i,t}^{down} + B_{i,t}^{up} + H_{i,t} + S_{i,t} \right),$$

where $B_{i,t}^{down}$ and $B_{i,t}^{up}$ are client-specific downlink/uplink bytes, $H_{i,t}$ is protocol metadata and header cost, and $S_{i,t}$ is security or secure-aggregation overhead. If compression is used, model bytes can be replaced by

$$B_{model} = \frac{b_{param} d}{r_{comp}},$$

where b_{param} is bits per parameter and r_{comp} is the compression ratio. Future work can compare dense 32-bit FedAvg against 16-bit quantization, 8-bit quantization, top-k sparsification, and delta-only model updates. This would make the LP shadow-price analysis more meaningful because the communication budget would reflect realistic bandwidth tradeoffs rather than only dense model size.

10.2 Making Differential Evolution More Efficient

The professor's concern is correct: every objective evaluation in differential evolution requires a training run. In this project the GA objective trains a federated model for a candidate tuple (E, K, η), where E is local epochs, K is clients per round, and η is the learning rate. That makes the search expensive because the objective is not a cheap mathematical function; it is an entire FL experiment.

Several strategies can make this smarter:

- Multi-fidelity search: evaluate many candidates using fewer rounds, fewer clients, or fewer seeds, then promote only strong candidates to full early-stopped runs.
- Successive halving or Hyperband: stop weak configurations early and allocate more training budget only to promising ones.
- Surrogate-assisted search: train a regression model that predicts loss or accuracy from hyperparameters and communication cost, then use full FL training only for the most promising predicted candidates.
- Warm-starting: initialize nearby candidate configurations from already trained checkpoints instead of restarting all candidates from scratch.
- Caching: reuse previous results when differential evolution proposes repeated or nearly repeated rounded values of (E, K, η).
- Parallel evaluation: evaluate independent candidates concurrently across CPU cores or devices.
- Early rejection: terminate a candidate if its first few rounds are clearly worse than the current best candidate under both accuracy and communication.

The current implementation already uses a partial version of this idea by running the search on a smaller budget and then validating the selected GA configuration across the full 10-seed early-stopped setting. A stronger final version would use a formal multi-fidelity pipeline: cheap screening, medium-budget confirmation, and full-budget validation.

10.3 Extension to Non-Convex DNN Settings

This project already uses a small ReLU neural network, so the empirical training problem is non-convex. However, the theoretical parts are easier to state in convex or smooth settings. In a non-convex deep neural network setting, the goal usually changes from proving convergence to a global optimum to showing convergence toward a stationary point, for example where

$$\mathbb{E}\|\nabla F(w)\|^2$$

becomes small.

The approach can be extended to larger DNNs as follows:

- Replace convex optimality claims with stationarity-based convergence diagnostics.
- Track gradient norms, update norms, client drift, and loss decrease instead of assuming global optimality.
- Use FL methods designed for non-IID non-convex optimization, such as FedProx, SCAFFOLD, FedAdam, or FedYogi.
- Treat the LP communication problem as an outer resource-allocation layer. Even if DNN training is non-convex, communication budgets, client-selection policies, and shadow prices can still be analyzed with convex approximations.
- Treat CVaR fairness as a robust empirical objective rather than a guarantee of convex optimality.
- Validate all conclusions statistically across seeds because non-convex training is sensitive to initialization and client sampling.

Thus, the practical extension is not to claim global convergence for DNNs. Instead, the report should frame the method as an empirical non-convex FL system supported by stationarity diagnostics, communication-aware optimization, and repeated-seed validation.

11 Limitations

This remains a simulated federated learning system on one machine, not real distributed infrastructure. Communication is estimated from parameter counts rather than measured network traffic. CVaR weighting is a practical fairness heuristic, not a formal convergence theorem. GA validation is honest but expensive and can still be sensitive to search budget.